

Chương 15: Xử lý Chuỗi bằng RNN và CNN

Giảng viên: [Tên Giảng Viên]

Đại Học Đông Á

Ngày 10 tháng 8 năm 2026

Nội dung Chương 15

1. Dữ liệu Chuỗi và Mạng nơ-ron hồi quy (RNN)
2. Ứng dụng và Kiến Trúc Dựa Trên Chuỗi
3. Huấn luyện RNN: BPTT
4. Dự báo Chuỗi Thời Gian (Time Series)
5. Mạng RNN Sâu (Deep RNNs)
6. Kiến trúc Tế bào Nhớ: LSTM và GRU
7. WaveNet (Mạng CNN 1D cho Chuỗi)
8. Tổng kết

Khái niệm: Dữ liệu Chuỗi (Sequence Data) là gì?

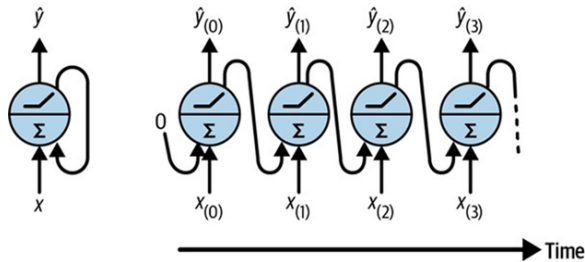
- Trong các chương trước, chúng ta xử lý dữ liệu nơi mỗi điểm dữ liệu là **độc lập và phân phối chuẩn (i.i.d)**.
- Tuy nhiên, trong **dữ liệu chuỗi**, tính thứ tự và sự phụ thuộc thời gian là cốt lõi.
- Ví dụ điển hình:
 - *Chuỗi thời gian kinh tế*: Giá cổ phiếu qua các ngày, GDP hằng năm.
 - *Dữ liệu y tế*: Điện tâm đồ (ECG), điện não đồ (EEG).
 - *Ngôn ngữ tự nhiên (NLP)*: Một câu là một chuỗi các từ có cấu trúc ngữ pháp.
 - *Âm thanh / Video*: Chuỗi các khung hình và sóng âm theo thời gian.

Giới hạn của Mạng Truyền thẳng (Feedforward Networks)

- **Mạng truyền thẳng (như MLP, CNN truyền thống):**
 - Kích thước đầu vào bị cố định (Fixed-size input).
 - Không có bộ nhớ (Memoryless): Mỗi lần nhận đầu vào mới, mạng quên hoàn toàn các đầu vào trước đó.
 - Không nắm bắt được phụ thuộc xa (Long-term dependencies) trừ khi nạp một lượng lớn đầu vào cùng lúc.
- Giải pháp: Cần một mạng có khả năng **lưu giữ trạng thái** của những gì nó đã thấy.

Kiến trúc Mạng RNN Cơ bản

- **Mạng Nơ-ron Hồi quy (RNN - Recurrent Neural Network)** được thiết kế để giải quyết vấn đề này.
- Một nơ-ron hồi quy đơn giản không chỉ nhận đầu vào từ dữ liệu $x_{(t)}$ tại bước t , mà còn nhận **trạng thái ẩn** $h_{(t-1)}$ từ bước trước đó.
- Mạng có thể xử lý chuỗi độ dài tùy ý vì nó thực hiện một phép tính lặp lại (recurrent) ở mỗi bước.



Hình 15-1: RNN cơ bản và phiên bản mở rộng

Trạng thái Ẩn (Hidden State) và Tính Toán Toán Học

- Tại mỗi bước thời gian t , nơ-ron RNN (hoặc lớp RNN) tính toán trạng thái tiếp theo như sau:

$$\mathbf{h}_{(t)} = \phi (\mathbf{W}_{hx}\mathbf{x}_{(t)} + \mathbf{W}_{hh}\mathbf{h}_{(t-1)} + \mathbf{b})$$

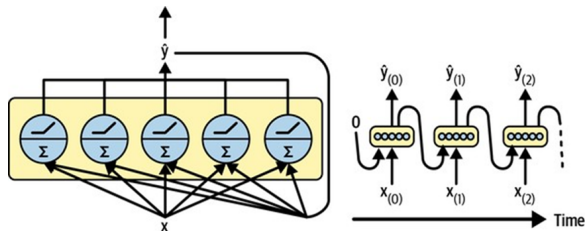
- Trong đó:
 - $\mathbf{x}_{(t)}$: Đầu vào tại bước t .
 - $\mathbf{h}_{(t-1)}$: Trạng thái ẩn từ bước $t - 1$ (mang thông tin "trí nhớ").
 - $\mathbf{W}_{hx}$: Trọng số cho đầu vào.
 - $\mathbf{W}_{hh}$: Trọng số cho trạng thái ẩn.
 - $\mathbf{b}$: Vector độ lệch (bias).
 - ϕ : Hàm kích hoạt (thường dùng $\tanh$ thay vì ReLU ở RNN chuẩn để tránh phát nổ giá trị).

Đặc điểm của Trọng Số trong RNN

- **Chia sẻ Trọng số (Weight Sharing):**
- Khác với mạng MLP nơi mỗi lớp có bộ trọng số riêng, RNN sử dụng **cùng một bộ ma trận trọng số $\mathbf{W}_{hx}$, $\mathbf{W}_{hh}$** cho tất cả các bước thời gian t .
- Lợi ích:
 - Giảm số lượng tham số đáng kể, tránh quá khớp (overfitting).
 - Có khả năng tổng quát hóa cho các chuỗi có độ dài chưa từng thấy trong tập huấn luyện.

Lớp RNN (RNN Cell Layer)

- Hình 15-2 biểu diễn một **lớp nơ-ron RNN** (RNN Layer) thay vì chỉ một nơ-ron đơn.
- Thay vì nhận một đặc trưng, mạng nhận một vector đặc trưng $\mathbf{x}_{(t)}$.
- Lớp này trả về vector đầu ra $\mathbf{y}_{(t)}$ (thường chính là $\mathbf{h}_{(t)}$).
- Ký hiệu vector giúp mở rộng sang phép tính ma trận cho **Mini-batch** (nhiều chuỗi cùng lúc).



Hình 15-2: Lớp RNN qua thời gian

Phân loại Bài toán xử lý Chuỗi

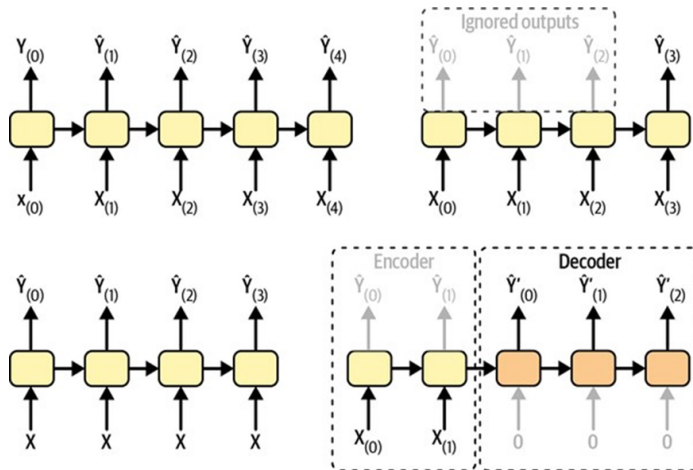
Tùy thuộc vào yêu cầu đầu vào / đầu ra, ta có các loại bài toán:

- ➊ **Sequence-to-Sequence (Chuỗi sang Chuỗi đồng bộ):** Đầu vào N bước, xuất N bước. (Dự báo thời tiết liên tục).
- ➋ **Sequence-to-Vector (Chuỗi sang Vector):** Đầu vào N bước, chỉ xuất 1 kết quả ở cuối chuỗi. (Phân loại văn bản, Sentiment analysis).
- ➌ **Vector-to-Sequence (Vector sang Chuỗi):** Đầu vào 1 giá trị, sinh ra chuỗi độ dài N . (Tạo chú thích ảnh - Image Captioning).
- ➍ **Encoder-Decoder (Delayed Seq-to-Seq):** Nhận chuỗi N , dịch qua chuỗi M . (Dịch máy, Trả lời câu hỏi).

Minh Họa 4 Dạng Kiến Trúc RNN Cốt Lõi

Ghi chú Hình 15-4:

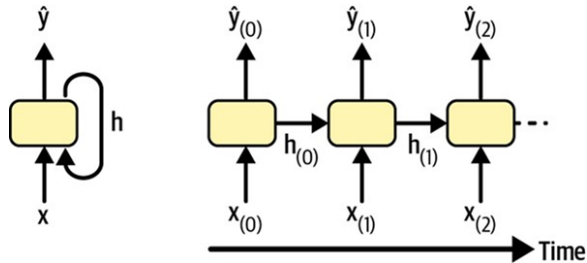
- *Trên trái:* Seq-to-Seq (đồng bộ).
- *Trên phải:* Seq-to-Vec. Đầu ra chỉ quan tâm ở bước cuối.
- *Dưới trái:* Vec-to-Seq. Đầu vào duy nhất ở $t = 0$.
- *Dưới phải:* Mã hóa - Giải mã.



Hình 15-4: Các dạng Seq2Seq, Seq2Vec, Vec2Seq

Ứng Dụng: Mạng Chuỗi Sang Chuỗi Dự Báo Trễ

- **Hình 15-3** mô tả kiến trúc dùng nhiều nhất cho Time Series Forecasting (Dự báo chuỗi thời gian).
- Kỳ vọng mạng dự đoán tương lai $y_{(0)}, y_{(1)} \dots$ dựa trên quá khứ $x_{(0)}, x_{(1)}, \dots$
- Đầu ra $y_{(t)}$ tại thời điểm t là kết quả dự báo cho $x_{(t+1)}$, do đó mạng bị trễ 1 bước (shifted by 1 step).



Hình 15-3: Seq-to-Seq trễ trong dự báo

Huấn luyện mạng Neural truyền thống (Nhắc lại)

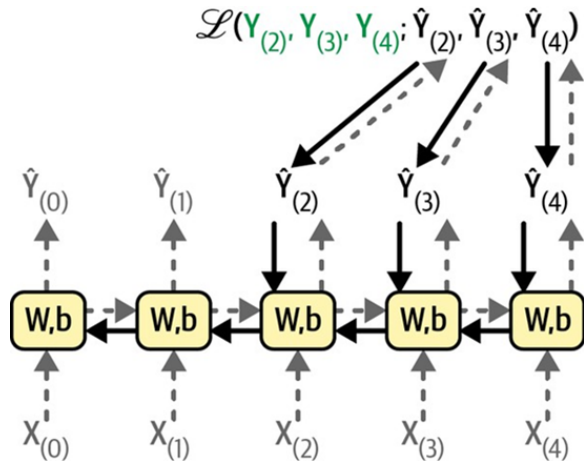
- Mạng truyền thẳng được huấn luyện bằng **Lan truyền ngược (Backpropagation)**:
 - ① Lan truyền xuôi (Forward pass) để tính hàm mất mát (Loss).
 - ② Tính Gradient (đạo hàm) của Loss theo các trọng số từ lớp cuối về lớp đầu bằng quy tắc chuỗi (Chain rule).
 - ③ Cập nhật trọng số bằng Gradient Descent.
- Đối với RNN, chuỗi thời gian $t = 1 \dots T$ làm cấu trúc mạng giống như một mạng rất sâu T lớp. Cần một biến thể đặc biệt.

Backpropagation Through Time (BPTT)

- **Lan truyền ngược qua thời gian (BPTT)** là kỹ thuật huấn luyện RNN:
- **Bước 1 (Mở cuộn - Unrolling):** Tạm thời xem RNN như một mạng Feedforward khổng lồ với T lớp (nếu chuỗi dài T).
- **Bước 2 (Lan truyền xuôi):** Tính Loss trên mọi bước thời gian (hoặc chỉ bước cuối).
- **Bước 3 (Lan truyền ngược):** Gradient lan truyền ngược dọc theo chiều thời gian từ $t = T$ về $t = 0$.
- Tổng Gradient cho trọng số mạng là tổng của các Gradient tại mọi bước thời gian.

Cơ chế Hoạt động của BPTT (Minh họa)

- **Hình 15-5** minh họa quá trình BPTT.
- Mũi tên liền nét dày mô tả luồng Gradient ngược từ hàm mất mát L .
- Do $\mathbf{W}$ được chia sẻ ở mọi t , gradient cuối cùng để cập nhật $\mathbf{W}$ là tổng $\sum \nabla \mathbf{W}^{(t)}$.
- Phép tính này cực kỳ đắt đỏ và tốn RAM nếu độ dài chuỗi rất lớn.



Hình 15-5: BPTT - Lan truyền ngược qua thời gian

Vấn Đề: Vanishing Gradients (Suy thoái Gradient)

- **Nguyên nhân:** Trong BPTT, đạo hàm phải được nhân với ma trận trọng số W_{hh} liên tục T lần (qua T bước thời gian).
- $\nabla h_{(0)} \propto (W_{hh})^T \nabla h_{(T)}$
- Nếu giá trị riêng lớn nhất của $W_{hh} < 1$, gradient tiến dần về 0 theo hàm số mũ.
- **Hậu quả:** Mạng không thể học được phụ thuộc xa (Ví dụ: Từ ở đầu câu văn quá dài không có ảnh hưởng gì tới dự đoán ở cuối câu). Trí nhớ mạng bị "ngắn hạn".

Vấn Đề: Exploding Gradients (Bùng nổ Gradient)

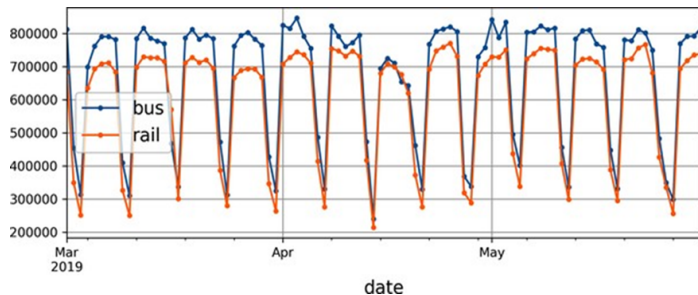
- Ngược lại, nếu giá trị riêng lớn nhất của $W_{hh} > 1$, gradient bị nhân lên theo cấp số nhân và phát nổ thành NaN hoặc vô cực.
- **Hậu quả:** Cập nhật trọng số quá lớn, làm mất ổn định hoàn toàn mạng, Loss dao động điên cuồng.
- **Giải pháp cho bùng nổ:**
 - **Gradient Clipping:** Cắt cụt gradient nếu nó vượt quá một ngưỡng nhất định trước khi cập nhật.
 - Thường xuyên được áp dụng mặc định trong các Framework như Keras/PyTorch.

Bài toán Dự báo Chuỗi Thời Gian Thực Tế

- Chúng ta sẽ xem xét dữ liệu thực tế để hiểu tại sao cần RNN phức tạp.
- Dự báo chuỗi thời gian (Time series forecasting) thường phải xử lý:
 - Nhiễu trắng (White noise) ngẫu nhiên.
 - Tính thời vụ (Seasonality) - Các mẫu lặp lại đặn (Ngày, tuần, tháng).
 - Xu hướng (Trend) - Tăng trưởng hoặc suy giảm chậm trong thời gian dài.
- Tiền xử lý dữ liệu để loại bỏ những yếu tố này đôi khi hiệu quả hơn để mạng nơ-ron tự học từ đầu.

Ví dụ: Lượng hành khách xe buýt (Chicago)

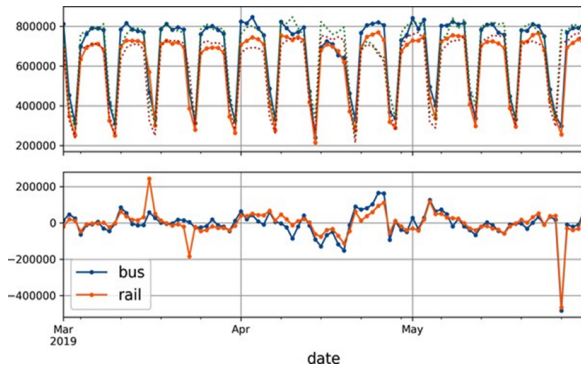
- Tập dữ liệu lượng hành khách hàng ngày ở Chicago (2019).
- Trục hoành là Ngày, trục tung là số hành khách.
- **Hình 15-6** cho thấy các biến động liên tục và nhịp nhàng. Mỗi "khôi" lỏm chớm tương ứng với một tuần làm việc (cao điểm) và cuối tuần (giảm sút).



Hình 15-6: Lượng hành khách hằng ngày ở Chicago

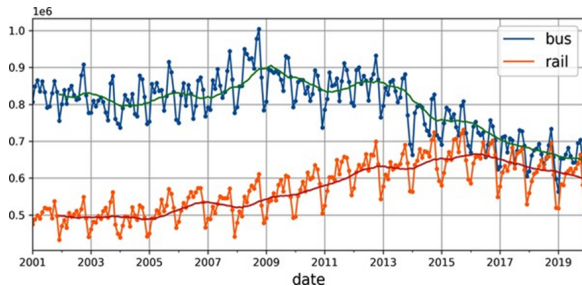
Kỹ thuật Tiền Xử Lý: Sai phân (Differencing)

- Để giúp mạng dễ dàng học hơn, ta có thể khử **tính thời vụ hàng tuần** bằng phép tính sai phân trễ 7 ngày ($y_t - y_{t-7}$).
- **Hình 15-7** cho thấy: Biểu đồ trên mô tả chuỗi gốc so với chuỗi bị dịch (trễ).
- Biểu đồ dưới (chuỗi sai phân) mượt mà, stationary (dừng) và loại bỏ được nhiễu do khác biệt giữa ngày thường và cuối tuần.



Hình 15-7: Chuỗi thời gian trễ 7 ngày và phép sai phân

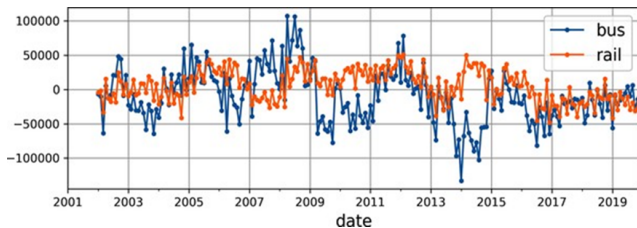
- Mở rộng ra 15 năm (2001-2015), **Hình 15-8** cho thấy rõ ràng các ngày lễ hội khiến lượng khách sụt giảm lớn hàng năm, và tổng thể hành khách biến động vĩ mô (Macro Trend).
- Để RNN mô hình hóa được điều này, nó cần "Trí nhớ Dài Hạn" vươn xa hàng trăm bước, cực kỳ khó nếu dùng SimpleRNN.



Hình 15-8: Tính thời vụ hằng năm và xu hướng dài hạn

Kỹ thuật Tiền Xử Lý: Sai phân Dài hạn

- Áp dụng sai phân 12 tháng liên tục.
- **Hình 15-9** cho thấy chuỗi kết quả cân bằng hơn quanh mốc 0.
- Mạng RNN hiện đại có thể tự phát hiện các trend này, nhưng trong thực tế, đưa vào chuỗi sai phân thường hội tụ nhanh và dự đoán chính xác hơn dữ liệu gốc hỗn độn.



Hình 15-9: Sự sai phân 12 tháng (Dài hạn)

Code: API Huấn luyện RNN cơ bản với Keras

Để xây dựng SimpleRNN trên Keras, dùng lớp mô phỏng nơ-ron hồi quy:

```
import tensorflow as tf
from tensorflow import keras

# Tao mang RNN 1 lop, 1 hidden unit
model = keras.Sequential([
    keras.layers.SimpleRNN(1, input_shape=[None, 1])
])

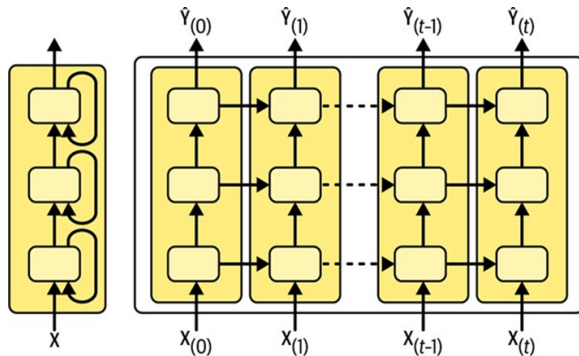
model.compile(loss="mse", optimizer="adam")
history = model.fit(X_train, y_train, epochs=20,
                    validation_data=(X_valid, y_valid))
```

Phân tích Code Keras RNN

- Tham số 1 trong `SimpleRNN(1)` là số lượng nơ-ron trạng thái ẩn (dimensionality of output space). Vì là dự đoán 1 bước tiếp theo (đại lượng vô hướng), nên chọn 1 nơ-ron.
- `input_shape=[None, 1]`:
 - `None`: Đại diện cho chiều dài của chuỗi (Time steps). Keras cho phép chuỗi dài ngắn tùy ý nếu ta đặt là `None`.
 - `1`: Số chiều (Features) tại mỗi bước thời gian. (Chỉ có 1 đặc trưng là lượng hành khách).
- Hàm Loss thường dùng cho bài toán dự báo chuỗi là Mean Squared Error (`mse`) hoặc Mean Absolute Error (`mae`).

Kiến trúc Mạng RNN Sâu

- Tương tự MLP Sâu, ta **xếp chồng (stacking)** các lớp RNN lên nhau tạo thành **Deep RNN**.
- Lớp dưới cùng nhận dữ liệu đầu vào. Đầu ra trạng thái ẩn $\mathbf{h}_{(t)}$ của lớp dưới làm đầu vào cho lớp trên tại **cùng thời điểm t** .
- **Hình 15-10** chỉ ra mũi tên lan truyền dọc theo cả 2 trục: Trục thời gian (ngang) và Trục không gian / Chiều sâu (dọc).



Hình 15-10: Mạng RNN sâu nhiều lớp

Code: Xây dựng Mạng RNN Sâu

- Yêu cầu quan trọng: Để lớp RNN dưới có thể đẩy chuỗi output lên lớp RNN trên, nó **BẮT BUỘC** phải bật `return_sequences=True`.

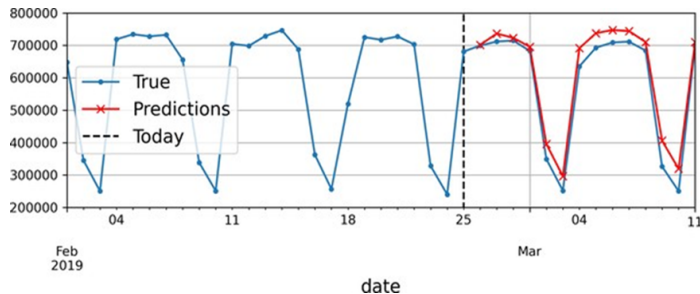
```
deep_model = keras.Sequential([
    # Lop RNN 1: Tra ve chuỗi cho lop 2
    keras.layers.SimpleRNN(20, return_sequences=True, input_shape=[None, 1]),
    # Lop RNN 2: Tra ve chuỗi cho lop cuối
    keras.layers.SimpleRNN(20, return_sequences=True),
    # Lop RNN cuối: Chỉ tra ve vector cuối cùng (Seq2Vec)
    keras.layers.SimpleRNN(1)
])
```

Dự báo nhiều bước (Multi-step Forecasting)

- Mạng trên chỉ dự báo $t + 1$. Để dự báo xa (VD: $t + 10$), có 2 cách:
 - ① **Cách 1 (Dự đoán tuần tự - Autoregressive):** Dự đoán $t + 1$, lấy giá trị đó gộp vào mảng đầu vào để đoán $t + 2$, lặp lại 10 lần. (Dễ tích lũy sai số).
 - ② **Cách 2 (Dự đoán đồng thời):** Đổi lớp cuối thành Dense(10) để bắt mạng xuất 10 giá trị cùng một lúc. Cách này thường ổn định hơn.

Kết quả dự báo đa bước của RNN Sâu

- **Hình 15-11** là kết quả dự báo một chuỗi nhân tạo nhiều bước đồng thời.
- Dấu $\times$ là dữ liệu đầu vào.
- Đường đứt nét là giá trị mục tiêu (Target).
- Dấu chấm tròn là giá trị dự đoán do mạng RNN sâu đưa ra.
- Mạng bám sát rất tốt đường đi tự nhiên của sóng dốc.



Hình 15-11: RNN Sâu dự báo đa bước khá chính xác

Vấn đề cốt lõi của SimpleRNN (Nhắc lại)

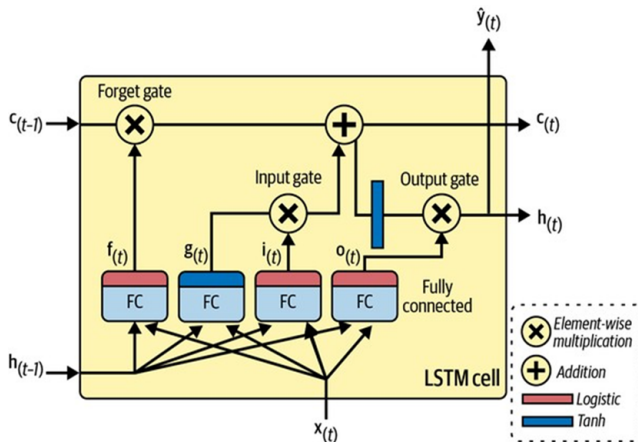
- SimpleRNN rất tồi trong việc nắm bắt **phụ thuộc xa (Long-Term Dependencies)**.
- Ví dụ câu NLP: "*Tôi sống ở Pháp từ nhỏ, sau 20 năm chuyển đến Mỹ, tôi vẫn nói trôi chảy tiếng [X].*". Mạng cần nhớ chữ "*Pháp*" ở quá khứ rất xa để đoán chữ "*Pháp*".
- Trọng số W nếu nhỏ thì Vanishing Gradient làm chữ "*Pháp*" bốc hơi sau vài bước thời gian.
- Để tạo ra "Trí nhớ Dài hạn", **Tế bào LSTM** được phát minh (Hochreiter và Schmidhuber - 1997).

Sơ lược về Tế bào LSTM (Long Short-Term Memory)

- Ý tưởng cốt lõi của LSTM là thay vì chỉ truyền một Trạng thái ẩn $h_{(t)}$, nó duy trì thêm một đường truyền phụ gọi là **Trạng thái tế bào (Cell State - $c_{(t)}$)**.
- $c_{(t)}$ giống như một đường ray cao tốc, chạy dọc xuyên suốt quá trình thời gian, cho phép thông tin truyền đi rất xa mà không suy giảm.
- Các cơ chế điều tiết (Gates - Cổng) kiểm soát việc ghi, đọc, xóa dữ liệu trên đường ray $c_{(t)}$ này.

Cấu trúc Tế bào LSTM chi tiết

- **Hình 15-12** là mô hình LSTM chuẩn. Đường ray ngang ở trên cùng là Trạng thái tế bào $c_{(t)}$. Đường ray dưới là Trạng thái ẩn $h_{(t)}$.
- Các phép toán tròn như $\otimes$, $\oplus$ là phép nhân phần tử và phép cộng vector.
- Cổng (Gate) sử dụng hàm kích hoạt Sigmoid (σ) (xuất giá trị $[0, 1]$) hoạt động như một cái "van": 0 là đóng hoàn toàn, 1 là mở hoàn toàn.



Hình 15-12: Cấu tạo bên trong Tế bào LSTM

Phân tích 3 Cổng của LSTM (Phần 1)

Dựa vào sơ đồ **Hình 15-12**:

- **1. Cổng Quên (Forget Gate - $f_{(t)}$):**

- Nhận vào $h_{(t-1)}$ và $x_{(t)}$, tính toán bằng Sigmoid.
- Quyết định phần nào của Trạng thái dài hạn $c_{(t-1)}$ sẽ bị "bỏ đi". (Ví dụ: Thấy chủ ngữ mới thì quên chủ ngữ cũ).

- **2. Cổng Cập nhật & Ghi (Input Gate - $i_{(t)}, g_{(t)}$):**

- $g_{(t)}$ tạo ra một vector giá trị ứng viên mới (dùng tanh).
- $i_{(t)}$ quyết định (bằng Sigmoid) mức độ quan trọng của các ứng viên này để cộng vào $c_{(t)}$ làm thông tin mới.

Phân tích 3 Cổng của LSTM (Phần 2)

- **Cập nhật Trạng thái Tế bào ($c_{(t)}$):**

$$c_{(t)} = f_{(t)} \otimes c_{(t-1)} + i_{(t)} \otimes g_{(t)}$$

(Quên đi quá khứ không cần thiết + Nhớ thêm thông tin mới).

- **3. Cổng Xuất (Output Gate - $o_{(t)}$):**

- Quyết định phần nào của Trạng thái Tế bào $c_{(t)}$ sẽ được đọc ra và đưa vào Trạng thái ẩn $h_{(t)}$.
- $o_{(t)} = \sigma(W_o[x_{(t)}, h_{(t-1)}])$
- $h_{(t)} = o_{(t)} \otimes \tanh(c_{(t)})$

Sử dụng LSTM trong Keras

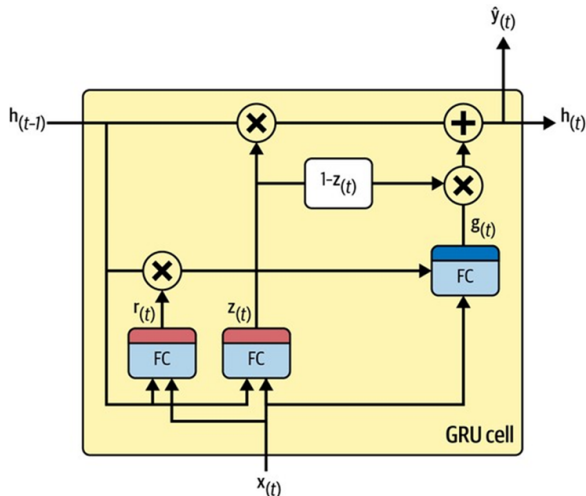
- Cực kỳ đơn giản, chỉ cần thay SimpleRNN bằng LSTM. Mọi công thức công thức phức tạp bên trên đã được Keras tối ưu bằng thư viện CUDNN dưới nền.

```
model = keras.Sequential([
    keras.layers.LSTM(20, return_sequences=True, input_shape=[None, 1]),
    keras.layers.LSTM(20),
    keras.layers.Dense(1)
])
```

- LSTM giải quyết tốt chuỗi dài khoảng hàng trăm bước thời gian, vượt xa SimpleRNN (chỉ hiệu quả với < 20 bước).

Tế bào GRU (Gated Recurrent Unit)

- GRU (2014) là một nỗ lực đơn giản hóa cấu trúc của LSTM.
- Hai điểm khác biệt chính:
 - GRU không có đường ray $c_{(t)}$ riêng biệt. Nó gộp chung Trạng thái ẩn $h_{(t)}$ và Trạng thái tế bào làm một.
 - Gộp Cổng Quên và Cổng Cập nhật thành một **Cổng Cập nhật (Update Gate - $z_{(t)}$)**. Nếu $z = 1$ thì lưu mới, nếu $z = 0$ thì giữ cũ.



Hình 15-13: Cấu tạo Tế bào GRU

So sánh LSTM và GRU

Đặc điểm	LSTM	GRU
Năm ra đời	1997	2014
Trạng thái duy trì	h_t và c_t	Chỉ h_t
Số lượng Cổng	3 (Quên, Nhập, Xuất)	2 (Cập nhật, Reset)
Tốc độ huấn luyện	Chậm	Nhanh hơn
Số lượng tham số	Lớn (Nặng RAM)	Thấp hơn ($\sim 25\%$ ít hơn)
Độ chính xác	Tốt với chuỗi rất phức tạp	Ngang ngửa LSTM ở đa số Data

- **Lời khuyên thực tiễn:** Bắt đầu thử nghiệm với GRU vì nó nhanh. Nếu thấy hiệu năng có vẻ bị thất sút, chuyển sang LSTM xem có cải thiện không.

Tại sao lại dùng CNN cho Dữ liệu Chuỗi?

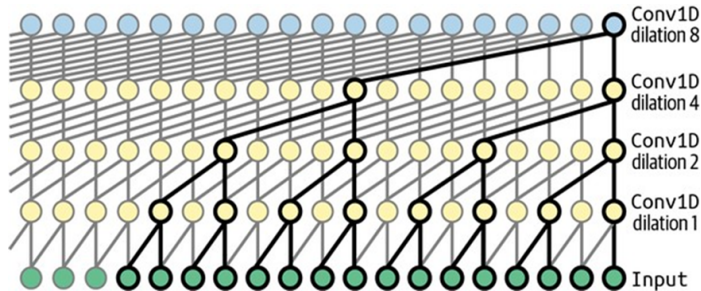
- RNN (ngay cả LSTM/GRU) có một điểm yếu chết người: **Tính toán tuần tự**.
- Bạn không thể tính bước thời gian $t = 100$ nếu chưa tính xong bước $t = 99$. Điều này khiến RNN **không thể chạy song song** trên phần cứng GPU một cách hiệu quả.
- Mạng Tích chập (CNN) với Kernel trượt thì độc lập với nhau, **hoàn toàn song song hóa** được.
- Ta có thể dùng **CNN 1D** (chỉ quét theo chiều dài thời gian) để trích xuất đặc trưng chuỗi.

Hạn chế của CNN 1D truyền thống

- Bộ lọc (Kernel) CNN 1D có kích thước nhỏ (VD: size 3).
- Nghĩa là mỗi nơ-ron lớp CNN 1 chỉ nhìn thấy 3 bước thời gian.
- Để nhìn được 100 bước thời gian (Trường thụ cảm - Receptive field lớn), ta phải xếp chồng hàng chục lớp CNN 1D, kết hợp với các lớp Pooling.
- Việc này rất nặng nề và làm mất đi thông tin chuỗi chi tiết. Cần một cơ chế quét khác!

Giải pháp đột phá: Kiến trúc WaveNet (2016)

- WaveNet do DeepMind phát triển, chuyên tạo giọng nói Audio. Âm thanh có tới 16,000 mẫu/giây $\Rightarrow$ Chuỗi cực kỳ dài!
- Thay vì CNN thường, nó sử dụng **Tích chập giãn nở (Dilated Convolutions)**.
- **Hình 15-14** minh họa: Tỷ lệ giãn (Dilation rate) tăng dần theo lũy thừa 2^n (1, 2, 4, 8...).



Hình 15-14: Kiến trúc WaveNet (Dilated Conv)

Cách hoạt động của Dilated Convolution

- Lớp 1 (rate=1): Kernel trượt từng ô liền kề (Giống hệt Conv1D thường).
- Lớp 2 (rate=2): Kernel bỏ qua 1 ô xen kẽ, giúp 1 nơ-ron bao phủ được khoảng cách 4 ô.
- Lớp 3 (rate=4): Kernel bỏ qua 3 ô xen kẽ, một nơ-ron nhìn được 8 ô.
- Nhờ tăng rate theo lũy thừa, mạng chỉ cần 10 lớp là đã có Trường thụ cảm bao phủ được **1024 bước thời gian** mà số lượng tham số vẫn cực kỳ nhỏ.

Code: Xây dựng WaveNet với Keras

Ta dùng vòng lặp để tạo các lớp Conv1D giãn nở:

```
model = keras.Sequential()
model.add(keras.layers.InputLayer(input_shape=[None, 1]))

# Các lớp tích chập giãn nở (1, 2, 4, 8) lặp lại
for rate in (1, 2, 4, 8) * 2:
    model.add(keras.layers.Conv1D(
        filters=20, kernel_size=2, padding="causal",
        activation="relu", dilation_rate=rate))

model.add(keras.layers.Conv1D(filters=10, kernel_size=1))
model.add(keras.layers.Dense(1)) # Lớp dự đoán
```

- Tham số padding="causal" cực kỳ quan trọng ở Time Series, nó chèn 0 vào bên trái để Kernel không bao giờ với tới thông tin của "tương lai".

Tổng Kết Chương 15 (Phần 1)

- **Sequence Data:** Đòi hỏi mô hình có bộ nhớ để theo dõi thứ tự.
- **RNN Cơ bản:** Chia sẻ trọng số qua các bước thời gian. Dùng Trạng thái ẩn $h_{(t)}$ để mang thông tin tới tương lai.
- **Ứng dụng RNN đa dạng:** Seq2Seq (Dự báo, Dịch), Seq2Vec (Phân tích cảm xúc), Encoder-Decoder (Chatbot).
- **BPTT (Backpropagation Through Time):** Mở cuộn mạng qua không gian thời gian. Dễ gây ra hiện tượng Gradient bốc hơi hoặc bùng nổ khi chuỗi quá dài.

Tổng Kết Chương 15 (Phần 2)

- **Tiền xử lý chuỗi (Differencing):** Rất hữu ích để loại bỏ tính thời vụ và xu hướng trước khi đưa vào mạng học sâu.
- **LSTM & GRU:** Thiết kế Tế bào thông minh với các "Cổng"(Gate) động đóng mở bằng Sigmoid, giúp bảo toàn Gradient đi xa, mang lại "Trí nhớ Dài Hạn".
- **CNN 1D & WaveNet:** Một hướng đi song song (không tuần tự), nhanh hơn RNN, dùng Dilated Convolutions để nắm bắt chuỗi độ dài hàng chục ngàn bước (vd: Âm thanh).